

KOMPUTASI PARALEL DAN TERDISTRIBUSI
Implementasi dan Pengujian Komputasi Paralel Menggunakan *OpenMP* dan
Sistem Terdistribusi Menggunakan *OpenMPI* untuk Menjumlahkan
Bilangan Akar



Dosen Pengampu:

Septian Geges, S.Kom., M.Kom.
(199109172020121004)

Disusun Oleh (Kelompok 3):

- | | |
|---------------------------|---------------|
| 1. Khairullah | 2430306030012 |
| 2. Hadi Setiawan | 2430205030016 |
| 3. Jona Surya Mapau | 2430305030026 |
| 4. Bintang Saputra Kahaya | 2430205030033 |
| 5. Ahmad Amin Badani | 2430205030017 |

JURUSAN TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS PALANGKA RAYA
2026

ABSTRAK

Perhitungan matematika seperti penjumlahan bilangan akar (*square root*) dari 1 sampai n memerlukan perulangan (*looping*) untuk mendapatkan hasil. Jika proses looping dilakukan secara *sekuensial* (berurutan satu-persatu) akan memerlukan waktu yang lama. Dengan menggunakan komputasi paralel, proses perhitungan bisa dilakukan secara bersamaan (*concurrent*) memanfaatkan beberapa *cores* sekaligus, sehingga waktu yang pemrosesan bisa lebih cepat. Arsitektur sistem menggunakan pola *master-worker* menggunakan OpenMPI sebagai *Message Passing Interface*, dimana 1 *master* mengelola beberapa *workers*. *Master* mengirimkan konfigurasi ke *workers*, sedangkan *workers* hanya perlu melakukan komputasi paralel menggunakan OpenMP dan mengembalikan hasil komputasi ke *master*. Sistem memiliki rata-rata *serial fraction* $f=0.05$, terdapat 5% kode yang tidak bisa dijalankan secara paralel. Hasil pengujian menunjukkan *speedup* mendekati ideal ketika menggunakan 2 - 6 *cores*, pada 6 *cores* *speedup* 5.58x dan efisiensi 93%. Prediksi *amdahl* mulai terlihat ketika menggunakan 8 *cores*, *speedup* $S(8)=5.83$ dengan $S_amdahl(8)=5.92$, efisiensi menurun signifikan menjadi 73%. Komputasi paralel terdistribusi terbukti mempercepat waktu pemrosesan secara signifikan, konfigurasi maksimal 3 *workers* dan 4 *cores* per *worker* (total 12 *cores*) menghasilkan waktu eksekusi 4.32 detik untuk 5 miliar angka, dibandingkan 33.93 detik pada eksekusi serial dengan 1 *worker* dan 1 *core*.

Kata kunci: Penjumlahan Bilangan Akar, *Looping*, Komputasi Paralel, Sistem Terdistribusi

BAB I

PENDAHULUAN

A. Latar Belakang

Perhitungan matematika seperti penjumlahan bilangan akar (*square root*) dari 1 sampai n memerlukan perulangan (*looping*) untuk mendapatkan hasil (*output*). Jika proses *looping* dilakukan secara sekuensial (berurutan satu-persatu) akan memerlukan waktu yang lama. Dengan menggunakan komputasi paralel, proses perhitungan bisa dilakukan secara bersamaan (*concurrent*) memanfaatkan beberapa *threads/cores* sekaligus, sehingga waktu pemrosesan bisa lebih cepat. Proses perhitungan juga bisa dibagi ke beberapa komputer/*worker* (menggunakan *virtual machine/docker container*), sehingga sumber daya yang digunakan tidak dibebankan pada 1 komputer saja.

B. Rumusan Masalah

1. Bagaimana mengimplementasikan komputasi paralel untuk penjumlahan bilangan akar untuk mempercepat waktu pemrosesan, menggunakan OpenMP?
2. Bagaimana mengimplementasikan sistem terdistribusi untuk membagi proses ke beberapa *worker* sekaligus, menggunakan OpenMPI?
3. Bagaimana analisis perbandingan efisiensi dan *speedup* saat hanya menggunakan 1 *thread/cores* hingga n *threads/cores*?

C. Tujuan

1. Memahami cara kerja komputasi paralel menggunakan *OpenMP*
2. Memahami cara kerja sistem terdistribusi menggunakan *virtual machine/docker container* untuk pembagian tugas/proses
3. Mengetahui *speedup* dan efisiensi dari proses komputasi.

D. Batasan

Program yang dibuat terbatas pada penjumlahan bilangan akar, program dibuat menggunakan C++. Simulasi sistem terdistribusi menggunakan *container*, dilakukan menggunakan 1 komputer dengan 3 *container* yang saling terisolasi.

BAB II

TINJUAN PUSTAKA

A. Pengertian Komputasi Paralel

Komputasi paralel adalah sebuah cara untuk menyelesaikan permasalahan dengan memecahnya menjadi beberapa bagian (disebut *instructions*), yang masing-masing bagian diselesaikan secara bersamaan oleh beberapa *processor* sekaligus [1]. Terdapat beberapa alasan penggunaan komputasi paralel, 2 diantaranya adalah *speed* dan *concurrency*. Waktu yang digunakan lebih singkat, program yang dibuat menggunakan komputasi paralel lebih cepat terselesaikan jika dibandingkan komputasi serial yang hanya menggunakan 1 *processor*. *Concurrency* secara sederhana adalah melakukan beberapa hal sekaligus, komputasi paralel dapat melakukan beberapa eksekusi yang berbeda dalam waktu bersamaan, berbeda dengan komputasi serial hanya bisa melakukan 1 proses dalam 1 waktu.

Pada komputasi paralel, terdapat beberapa istilah khusus:

1. *Node*: sebuah mesin/komputer
2. *CPU (processor)*: perangkat keras yang ada pada komputer
3. *Core*: perangkat keras bagian dari CPU yang terdiri dari beberapa *cores*, setiap *cores* mampu melakukan tugas secara independen, terkadang diartikan sama dengan CPU (*core = cpu*)
4. *Process*: sesuatu yang dijalankan/berjalan (*running*)
5. *Shared memory*: konsep dimana seluruh *cores* memiliki akses ke sebuah *memory* yang sama, digunakan pada *single node*
6. *Threads*: unit logika (*logical unit*) yang memungkinkan 1 *core* untuk mengelola beberapa instruksi untuk meningkatkan efisiensi, 1 *core* memiliki setidaknya 1 thread, beberapa *threads* mengakses sumber daya yang sama pada sebuah core (*multi-threading*) [2] [3].
7. *Massive parallel*: sebuah proses yang menggunakan banyak *cores* (ratusan hingga ribuan)
8. *Embarrassingly parallel*: penyelesaian masalah dari beberapa tugas (*tasks*) yang independen, dengan sedikit atau tidak ada koordinasi [4].

B. Klasifikasi Arsitektur Komputer menggunakan *Flynn's Taxonomy*

Komputasi paralel dapat diklasifikasikan melalui beberapa cara, salah satu yang banyak digunakan adalah *Flynn's Taxonomy*, diperkenalkan tahun 1996 oleh Michael J. Flynn. *Flynn's Taxonomy* mengelompokkan arsitektur *multi-processor* pada komputer berdasarkan dua dimensi: *instruction stream* dan *data stream*, yang masing-masing dimensi memiliki dua kemungkinan: *single* atau *multiple* [5].

1. *Single Instruction, Single Data* (SISD): sebuah instruksi digunakan untuk memproses sebuah data. Contoh $x = I + I$;
2. *Single Instruction, Multiple Data* (SIMD): sebuah instruksi digunakan untuk memproses beberapa data. Contoh $x = [1, 2, 3, 4, 5]$; dan untuk seluruh *chunks* x lakukan komputasi $y += (2 * x)$
3. *Multiple Instruction, Single Data* (MISD): beberapa instruksi digunakan untuk memproses sebuah data, jarang digunakan.
4. *Multiple Instruction, Multiple Data* (MIMD): beberapa instruksi digunakan untuk memproses banyak data, implementasi komputasi paralel

C. Amdahl's Law

Amdahl's law diproposisikan oleh Gene Amdahl 1967, mendefinisikan teori peningkatan kinerja (*speedup*) dibatasi (*limited*) oleh sebagian (*fraction*) tugas yang tidak bisa diparalelisasi [6]. Seberapapun banyaknya *cores* yang digunakan untuk tugas paralel, kecepatan pemrosesannya akan selalu dibatasi oleh persentase tugas yang harus dilakukan secara serial (tidak bisa paralel).

Rumus yang digunakan adalah $Speedup(n) = \frac{1}{(1 - P) + \frac{P}{n}}$; dimana P adalah persentase tugas yang dapat dikerjakan secara paralel dan n adalah jumlah *cores* [7], terkadang notasi n diganti dengan $p = processor$. Rumus yang sama juga bisa ditulis $Speedup(n) = \frac{1}{f + \frac{(1-f)}{n}}$; dimana f adalah persentase tugas yang hanya bisa dikerjakan secara serial. Misalkan terdapat sebuah program yang 60% (0.60) dijalankan secara paralel dan menggunakan

2 cores, maka $Speedup(n) = \frac{1}{(1 - 0.60) + \frac{0.60}{2}} = \frac{1}{0.40 + 0.30} = \frac{1}{0.70} = 1.43$.

Dari hasil yang didapat, kecepatan program bisa dipercepat sebanyak 1.43x. Jika ditambahkan 1 core, sehingga total nilai n menjadi 3 cores, maka

$Speedup(n) = \frac{1}{(1 - 0.60) + \frac{0.60}{3}} = \frac{1}{0.40 + 0.20} = \frac{1}{0.60} = 1.67$. Terlihat bahwa

kecepatannya bertambah, namun antara 2 ke 3 cores, terlihat penurunan kecepatan, dari 1 ke 2 cores dipercepat 43% ($1.43 - 1.0$), sedangkan dari 2 ke 3 cores dipercepat hanya 24% ($1.67 - 1.43$). Jika terus ditambahkan hingga 60 cores bahkan sampai tidak terbatas, peningkatan kinerja yang didapat hanya bisa maksimal 2.43x.

D. Pengertian Sistem Terdistribusi

Sistem terdistribusi adalah sekumpulan perangkat komputasi yang saling terhubung sehingga terlihat sebagai sebuah sistem tunggal [8]. Sistem terdistribusi menggunakan *distributed memory*, dimana masing-masing perangkat komputasi (baik *processor* maupun komputer utuh) mengakses *memory* tersendiri sehingga menggunakan *message* untuk saling berkomunikasi. Berbeda dengan komputasi paralel yang menggunakan *shared memory*, mengakses langsung ke *memory* yang sama.

Terdapat 2 karakteristik dari sistem terdistribusi: perangkat komputasi (biasa disebut *node*) dan terlihat sebagai sistem tunggal. Perangkat komputer dapat berupa perangkat keras (contoh: komputer, laptop, dan server) maupun perangkat lunak (contoh: *virtual machine* dan *container*). Sedangkan, terlihat sebagai sistem tunggal dapat diartikan sebagai sebuah *layer* yang terlihat bagi pengguna seolah-olah sebagai sistem tunggal.

E. Arsitektur Sistem Terdistribusi Master-Worker

Arsitektur *Master-Worker* (atau dikenal juga sebagai *Master-Slave*) merupakan salah satu model struktur pengendalian yang paling umum digunakan dalam komputasi paralel dan sistem terdistribusi. Menurut riset yang dikembangkan dalam model performa aplikasi hibrida MPI + OpenMP, arsitektur ini memisahkan peran komponen sistem menjadi dua entitas utama

dengan fungsi yang kontras namun saling melengkapi, yaitu sebuah *node* pengendali (*Master*) dan beberapa *node* pemroses (*Workers*) [9].

Master node bertanggung jawab atas koordinasi sistem, yang meliputi pembagian beban kerja (*workload partitioning*), distribusi potongan tugas (*task/chunk distribution*), sinkronisasi, serta pengumpulan kembali hasil perhitungan parsial dari setiap komponen. Di sisi lain, *worker nodes* beroperasi secara pasif untuk menerima potongan tugas dari *master*, mengeksekusi komputasi secara mandiri menggunakan sumber daya lokal, dan mengembalikan hasil akhir ke *master* tanpa melakukan komunikasi antar sesama *worker*.

Dalam implementasi modern berkinerja tinggi (*high-performance computing*), model *Master-Worker* sering kali diintegrasikan secara hibrida menggunakan kombinasi *Message Passing Interface* (MPI) dan OpenMP. Protokol MPI digunakan oleh *master* untuk mendistribusikan data antar *node* melalui jaringan terdistribusi (*distributed memory*), sedangkan OpenMP dimanfaatkan di dalam masing-masing *worker node* untuk mengeksekusi tugas tersebut secara paralel *multithreading* memanfaatkan seluruh *core prosesor* yang tersedia (*shared memory*).

Terdapat beberapa pilihan komunikasi pada MPI [10]

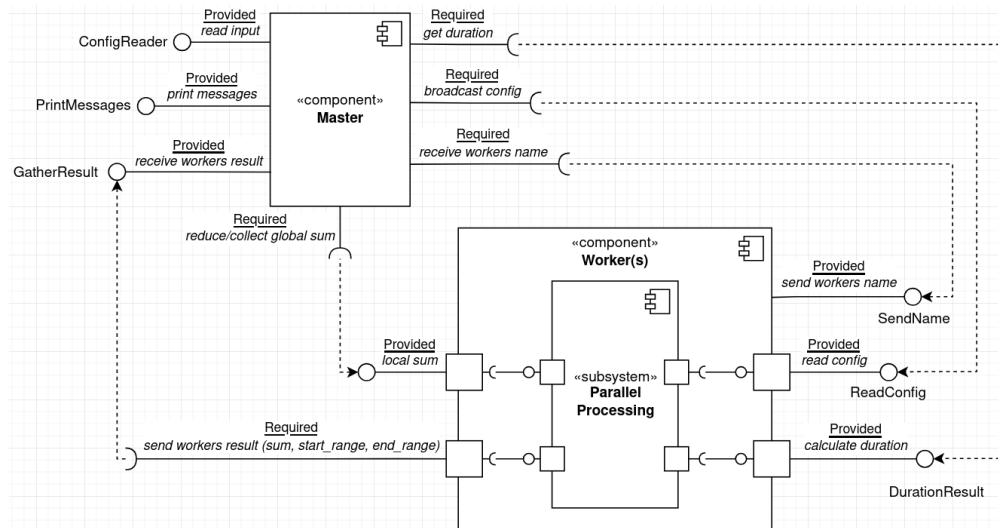
1. *Point-to-point*: mengirimkan data dari satu komputer sebagai pengirim ke tepat satu komputer lain sebagai penerima, contoh *MPI_Send* dan *MPI_Recv*
2. *Collective*: melibatkan seluruh komputer yang terhubung
 - a. *Broadcast*: satu komputer mengirimkan ke seluruh komputer lain.
Contoh: P0 = [1] menjadi P0 = [2]; P1 = [1]; P2 [2];
 - b. *Scatter-Gather*: *scatter* digunakan untuk membagi data secara rata dari satu komputer ke seluruh komputer lain, sedangkan *gather* digunakan untuk mengumpulkan seluruh data ke satu komputer
Contoh *scatter*: P0 = [1, 2, 3] menjadi P0 = [1]; P1 = [2]; P2 [3]
Contoh *gather*: P0 = [1]; P1 = [2]; P2 [3] menjadi P0 = [1, 2, 3]
 - c. *Reduce*: data dari seluruh komputer digabungkan/dijumlahkan ke satu komputer. Contoh: P0 = [1]; P1 = [2]; P2 [3] menjadi P0 = [6]

BAB III

DESAIN SISTEM

A. Gambaran Arsitektur Sistem

Arsitektur sistem menggunakan pola *master-worker*, dimana 1 *master* mendistribusikan tugas ke n *worker*. Arsitektur ini dipilih karena studi kasus yang diadaptasi berkaitan dengan perulangan (*looping*), spesifik menjumlahkan bilangan akar dari 1 sampai n . Sehingga input data dapat dibagikan ke masing-masing *workers*, *workers* memproses data secara paralel hanya pada *range* tertentu.



B. Komponen & Modul

Terdapat 2 *component* utama dan 1 buah *subsystem* pada sistem yang akan dibuat. Untuk kardinalitas dari *master* ke *worker* adalah *one-to-many* ($1 \dots *$), ada 1 *master* dan n *worker*.

1. Master: *Master* bertanggung jawab mengirimkan input konfigurasi ke setiap *workers*, menampilkan informasi dan hasil seluruh *workers*, serta tetap ikut memproses data.

Interface:

- 1) *Read input (provided)*: membaca input *total_numbers* dan *active_cores* dan menyimpannya sebagai konfigurasi

- 2) *Print messages (provided)*: menampilkan nama serta hasil perhitungan seluruh *workers*
 - 3) *Gather result (provided)*: menerima hasil masing-masing *workers*, termasuk informasi *range_start* dan *range_end*
 - 4) *Broadcast configuration (required)*: mengirimkan/menyiarkan konfigurasi ke seluruh *workers*
2. Worker(s): *workers* bertanggung jawab memproses data untuk mengubah seluruh data menjadi bentuk akar (*square root*) secara paralel serta mengirimkan informasi dan hasil

Interface:

- 1) *Send information (provided)*: mengirimkan nama *workers*
- 2) *Read config (provided)*: membaca konfigurasi untuk menentukan *range_start* dan *range_end*
- 3) *Local sum (provided)*: hasil perhitungan pada *workers* untuk digabungkan menggunakan MPI_Reduce
- 4) *Send local result (required)*: mengirimkan hasil perhitungan pada *workers*, adalah *local_sum*, *range_start*, dan *range_end*

Subsystem:

- *Parallel processing*: *Worker* memiliki *subsystem* untuk memproses data secara paralel menggunakan OpenMP, dari *range_start* sampai *range_end*. Jumlah *cores* disesuaikan konfigurasi oleh *active_cores*.

3. Durasi

Terdapat 2 durasi waktu, *global_duration* dan *local_duration*

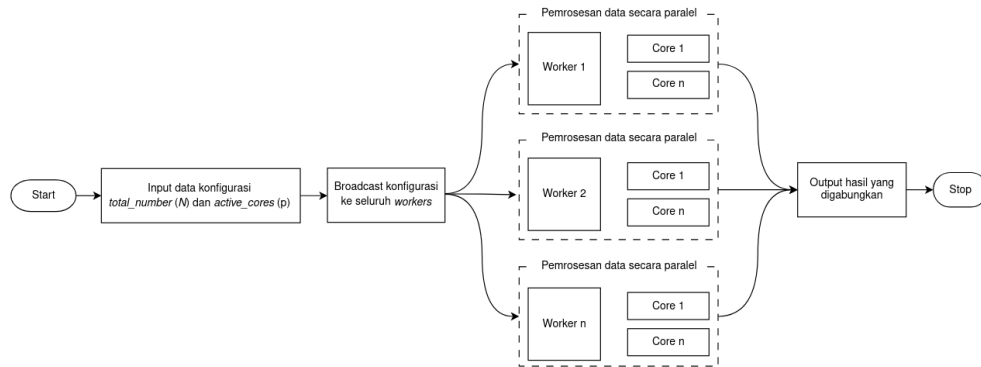
- 1) Durasi keseluruhan dari awal sampai akhir (*global_duration*)

Durasi keseluruhan hanya dihitung oleh *master*, mulai sebelum menampilkan pesan informasi *workers* sampai seluruh hasil *workers* selesai dikumpulkan. Didapatkan menggunakan *MPI_WTime()*

- 2) Durasi proses di *workers* (*local_duration*)

Durasi lokal dihitung oleh seluruh *workers*, perhitungan dilakukan hanya pada bagian proses paralel, mulai sebelum proses hingga selesai proses. Didapatkan menggunakan *omp_get_wtime()*

C. Alur Sistem



Misal terdapat 3 buah *nodes* (dengan *rank* 0 sampai 2) yang masing-masing memiliki 4 *cores*. Terdapat input *total_number* = 30 dan *active_cores* = 4.

1. Konfigurasi *total_number* dan *active_cores* dibaca oleh *master node* (*rank* = 0)
2. *Master* mengirimkan konfigurasi ke seluruh *worker nodes* (*rank* = 1 dan *rank* = 2)
3. Masing-masing *workers* menentukan rentang *start* dan *end* berdasarkan *rank*-nya, master juga ikut
 - a. *Master* (*rank* = 0): *start* = 1; *end* = 10
 - b. *Worker-1* (*rank* = 1): *start* = 11; *end* = 20
 - c. *Worker-2* (*rank* = 2): *start* = 21; *end* = 30
4. Masing-masing *nodes* memproses secara paralel dari rentang yang sudah didapat, menggunakan 4 *cores*
 - a. *Master* (*rank* = 0): *local_sum* = 22.468278
 - b. *Worker-1* (*rank* = 1): *local_sum* = 39.197700
 - c. *Worker-2* (*rank* = 2): *local_sum* = 50.416867
5. Hasil dikumpulkan ke *master*: $global_sum = 22.468278 + 39.197700 + 50.416867 = 112.082845$

BAB IV

IMPLEMENTASI

A. Setup Environment Docker/Podman Container

Lingkungan pengembangan yang digunakan adalah *container based*, dengan *image* Ubuntu:24.04. *Container* dipilih karena *lightweight* jika dibandingkan dengan *virtual machine*, memanfaatkan *isolated environment* secara tersendiri tanpa terpengaruh komputer *host*, dan mudah untuk direproduksi (*reproducible*) di semua komputer yang mendukung aplikasi *Docker* atau *Podman*. Terdapat 2 file yang diperlukan:

1. docker-compose.yml:

Digunakan untuk menjalankan dan mengelola 3 *containers* pada sebuah *isolated environment* sebagai *nodes*, 1 *container* berperan sebagai *master* dan 2 lainnya sebagai *workers*.

```
services:
  master:
    container_name: mpi_master # identifikasi container
    hostname: mpi-master # identifikasi nama host di dalam container
  worker1:
    container_name: mpi_worker1
    hostname: mpi-worker1
  worker2:
    hostname: mpi-worker2
    container_name: mpi_worker2
```

2. Dockerfile

Digunakan untuk membuat *image* yang merupakan gabungan instruksi untuk membangun masing-masing *containers*, seperti instalasi *gcc*, *openssh*, dan *openmpi*; konfigurasi *openssh* dan *openmpi*; konfigurasi *working directory*; sampai menjalankan layanan *ssh*.

```
FROM ubuntu:24.04
RUN apt-get update && apt-get install -y build-essential openmpi-bin
libopenmpi-dev openssh-server openssh-client
CMD ["/usr/sbin/sshd", "-D"]
```

B. Implementasi Kode Program

Kode program dibagi menjadi 2, *main.cpp* yang berisi program utama dan *parallel.h* yang berisi *function* komputasi paralel. Pada *main.cpp*, terdiri dari beberapa bagian:

1. Persiapan awal: *input loader* dan *broadcast config*

Pada persiapan awal, setiap *workers* akan mengirimkan nama sesuai dengan konfigurasi *docker-compose.yml* ke *master*. Selagi menunggu *workers* selesai mengirimkan nama, *master* akan memulai perhitungan durasi menggunakan *MPI_WTime()*, dilanjutkan dengan menampilkan nama dirinya sendiri dan menerima nama dari setiap *workers*. *Master* juga akan menyimpan konfigurasi *total_numbers* dan *active_cores* sesuai input *arguments* ketika program dijalankan. Diakhiri dengan *broadcast* konfigurasi ke seluruh *workers*.

```
int world_rank, world_size;
string current_node_name = MPI_Get_processor_name();
if (world_rank != 0) {
    MPI_Send(current_node_name, tag=0);
}
else {
    double start_global = MPI_Wtime();
    print('Master: ', current_node_name);
    string worker_node_name;
    for (int i = 1; i < world_size; i++) {
        MPI_Recv(worker_node_name, tag=0);
        print('Worker: ', i, worker_node_name);
    }
    current_config = {total_numbers: argv[1], active_cores: argv[2]}
}
MPI_Bcast(current_config);
```

2. Proses utama: *parallel compute*

Setiap *nodes* (*master* dan *workers*) melakukan perhitungan rentang *start* dan *end* berdasarkan *total_numbers* dari konfigurasi dan *world_rank* masing-masing *nodes*. Dilakukan proses komputasi paralel berdasarkan *active_cores*, penjumlahan bilangan akar dari *start* sampai *end*, sekaligus menghitung durasi komputasi menggunakan *omp_get_wtime()*.

```

long long chunk = current_config.total_numbers / world_size;
long long start = (world_rank * chunk) + 1;
long long end = (world_rank == world_size - 1) ?
current_config.total_numbers : start + chunk - 1;

double start_local = omp_get_wtime();
double local_sum = compute_parallel_sqrt_sum(start, end,
current_config.active_cores);
double end_local = omp_get_wtime();
double duration_local = end_local - start_local;

```

3. Penutupan proses: *reduce result* dan *output writer*

Hasil *local_sum*, *start_local*, *end_local*, dan *duration_local* dari masing-masing *workers* dikirimkan ke *master* untuk ditampilkan sebagai output. Jika output ditampilkan oleh *workers*, terkadang hasil dari *workers* muncul terlambat, sehingga lebih baik untuk seluruh output ditampilkan oleh *master*, *workers* hanya perlu fokus memproses dan mengirimkan hasil perhitungan. Hasil *local_sum* seluruhnya dijumlahkan menggunakan *MPI_Reduce()* pada variabel *global_sum*. Keseluruhan durasi proses dan *global_sum* ditampilkan oleh master.

```

if (world_rank != 0) {
    var local_result = {local_sum, start, end, duration_local};
    MPI_Send(local_result, tag=1);
}
double global_sum = 0;
MPI_Reduce(local_sum, global_sum);

if (world_rank == 0) {
    print('Master: ', local_sum, duration_local, 'from :', start, 'to: ', end)
    double worker_result;
    for (int i = 1; i < world_size; i++) {
        MPI_Recv(worker_result, tag=1);
        print(
            'Worker: ', i, worker_result.local_sum,
            worker.duration_local, 'from :', worker.start, 'to: ', worker.end
        )
    }
    end_global = MPI_Wtime();
    duration_global = end_global - start_global;
    print('Total sum: ', global_sum);
    print('Total duration: ', duration_global);
}

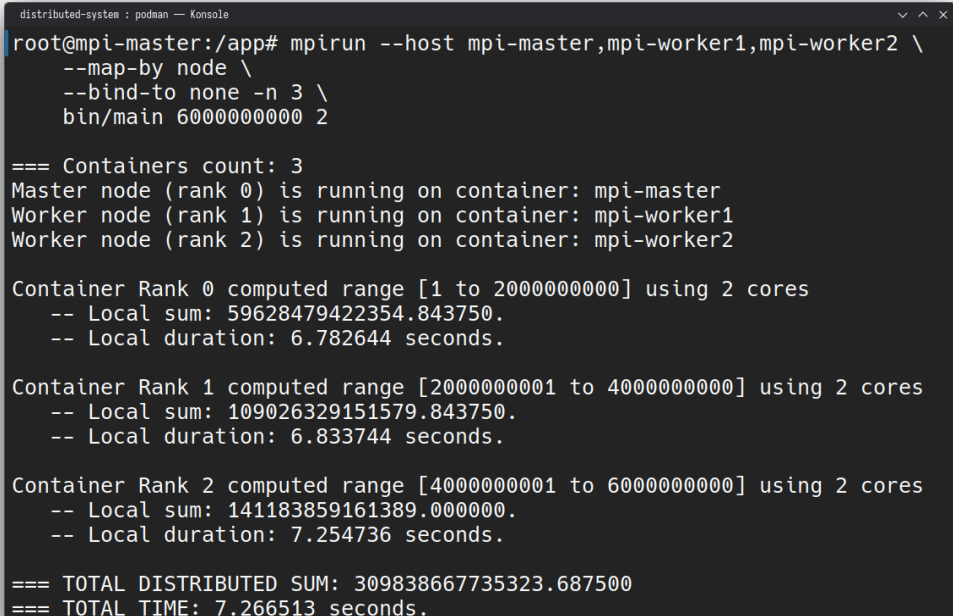
```

Untuk *parallel.h* hanya ada 1 *function*, *compute_parallel_sqrt_sum()* untuk melakukan penjumlahan bilangan akar sesuai nilai parameter *start*, *end*, dan *cores*. Di dalam *function* terdapat pengkondisian untuk memastikan tiga parameter yang diinputan valid. Komputasi paralel menggunakan *#pragma omp for reduction()*, *reduction* digunakan untuk menghindari *race condition* yang terjadi ketika mengupdate variabel yang sama (*shared variable*), variabel tersebut adalah *sum*.

```
double compute_parallel_sqrt_sum(int start, int end, int cores) {
    if (start > end || start < 0 || end < 0) { return 0; }

    double sum = 0;
    int used_cores = 1;
    if (cores >= 1 && cores <= 8) { used_cores = cores; }
    else if (cores > 8) { used_cores = 8; }

    #pragma omp parallel for reduction(+:sum) num_threads(used_cores)
    for (long long i = start_range; i <= end_range; i++) {
        sum += std::sqrt(i);
    }
    return sum;
}
```



```
distributed-system : podman -- Konsole
root@mpi-master:/app# mpirun --host mpi-master,mpi-worker1,mpi-worker2 \
--map-by node \
--bind-to none -n 3 \
bin/main 6000000000 2

=== Containers count: 3
Master node (rank 0) is running on container: mpi-master
Worker node (rank 1) is running on container: mpi-worker1
Worker node (rank 2) is running on container: mpi-worker2

Container Rank 0 computed range [1 to 2000000000] using 2 cores
-- Local sum: 59628479422354.843750.
-- Local duration: 6.782644 seconds.

Container Rank 1 computed range [2000000001 to 4000000000] using 2 cores
-- Local sum: 109026329151579.843750.
-- Local duration: 6.833744 seconds.

Container Rank 2 computed range [4000000001 to 6000000000] using 2 cores
-- Local sum: 141183859161389.000000.
-- Local duration: 7.254736 seconds.

=== TOTAL DISTRIBUTED SUM: 309838667735323.687500
=== TOTAL TIME: 7.266513 seconds.
```

BAB V

PENGUJIAN DAN ANALISIS PERFORMA

A. Spesifikasi Perangkat dan *Tools*

Perangkat: Laptop ThinkPad T14 Gen 1

Spesifikasi:

1. OS: Fedora Linux 43
2. CPU: AMD Ryzen 5 PRO 4650U (12) @ 2.10 GHz
3. GPU: AMD Radeon Vega Series / Radeon Vega Mobile Series
4. Memory: 16 GB SODIMM 3200 MT/s DDR4

Software: Docker/Podman Container

Docker Image: Ubuntu:24.04 dengan *gcc* 13.3.0 + *openmpi* 4.1.6 + *openssh*

B. Metodologi dan Hasil Pengujian

Pengujian dilakukan berupa uji kecepatan, dengan cara *running* program menggunakan *parameter* yang sama sebanyak 5 kali. Hasil yang diambil adalah median dari waktu keseluruhan (dari awal *MPI_Init*, proses paralel, sampai akhir *MPI_Finalize*).

1. Jumlah *workers*, minimal 1 dan maksimal 3
 2. Total angka (*total_numbers*), misal 5,000,000,000 (lima miliar)
 3. Total *cores* (*active_cores*), dari minimal 1 (serial) sampai 8 (total maksimal 12)
- Tabel pengujian *strong scaling*: $N = 5,000,000,000$

<i>Workers</i>	Total angka	<i>Cores</i> (per <i>workers</i>)	Total <i>cores</i>	Waktu (detik)
1	5,000,000,000	1	1	33.932419
1	5,000,000,000	2	2	16.937632
1	5,000,000,000	4	4	8.504499
2	5,000,000,000	1	2	17.014124
2	5,000,000,000	2	4	9.068681
2	5,000,000,000	4	8	5.807154

3	5,000,000,000	1	3	11.336493
3	5,000,000,000	2	6	6.071472
3	5,000,000,000	4	12	4.322645

- Tabel pengujian *weak scaling*: N per core = 1,000,000,000

<i>Workers</i>	Total angka	<i>Cores</i> (per <i>workers</i>)	Total <i>cores</i>	Waktu (detik)
1	1,000,000,000	1	1	6.764092
1	2,000,000,000	2	2	6.785999
1	4,000,000,000	4	4	6.847312
2	2,000,000,000	1	2	6.809764
2	4,000,000,000	2	4	6.827757
2	8,000,000,000	4	8	9.203509
3	3,000,000,000	1	3	7.266613
3	6,000,000,000	2	6	7.338209
3	12,000,000,000	4	12	10.414252

C. Analisis Pengujian

Dari hasil pengujian, dihitung *speedup* $S(p) = T_{seq} \div T_{par}(p)$ dan efisiensi $E(p) = S(p) \div p = T_{seq} \div (p * T_{par}(p))$ berdasarkan hasil *strong scaling*, untuk $N = 5,000,000,000$.

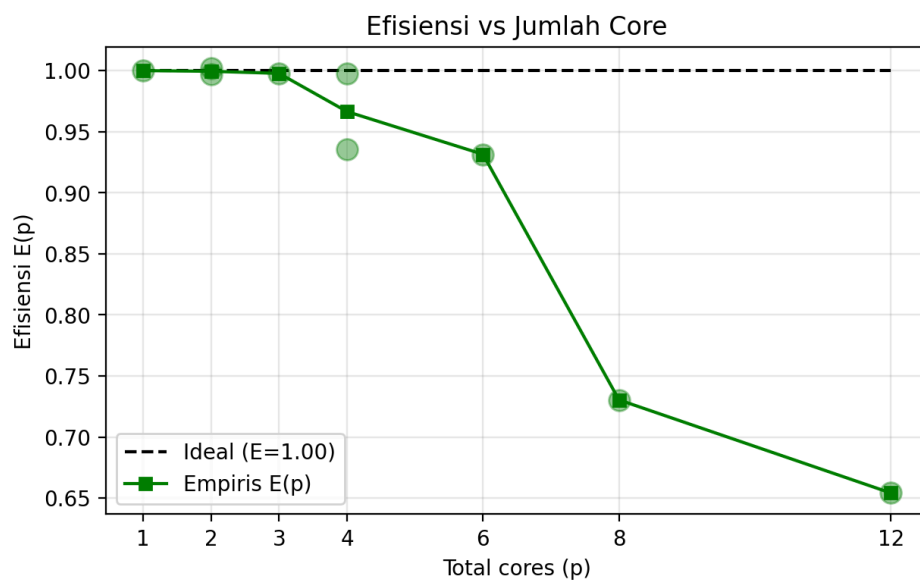
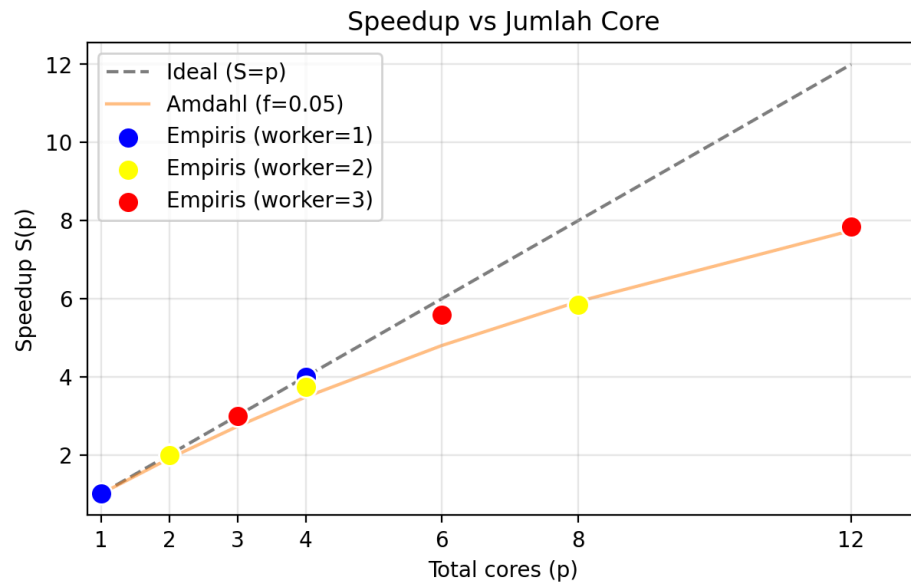
Workers	Total <i>cores</i>	Waktu (s)	S(p)	E(p)
1 (baseline)	1	33.932419	1.0000	100.00%
1	2	16.937632	2.0034	100.17% (superlinear)
1	4	8.504499	3.9901	99.75%
2	2	17.014124	1.9944	99.72%
2	4	9.068681	3.7419	93.55%

2	8	5.807154	5.8345	73.04%
3	3	11.336493	2.9923	99.74%
3	6	6.071472	5.5894	93.16%
3	12	4.322645	7.8503	65.42%

Sedangkan untuk menentukan performa *multi-node*, dilakukan perhitungan *throughput* $Throughput = N_{task} \div T_{total}$ guna mengetahui jumlah tugas/item yang diselesaikan/diproses per detik berdasarkan *weak scaling*, untuk $N = 1,000,000$ per *cores*

<i>Workers</i>	<i>Cores/worker</i>	Total N	Waktu (s)	Throughput (item per detik)
1	1	1 miliar	6.764092	147,840,664
1	2	2 miliar	6.785999	294,724,102
1	4	4 miliar	6.847312	584,171,558
2	1	2 miliar	6.809764	293,696,603
2	2	4 miliar	6.827757	585,844,369
2	4	8 miliar	9.203509	869,234,794
3	1	3 miliar	7.266613	412,847,149
3	2	6 miliar	7.338209	817,637,754
3	4	12 miliar	10.414252	1,152,488,336

Untuk analisis Amdahl, harus diketahui terlebih dahulu f atau persentase kode yang tidak bisa dilakukan secara paralel. Nilai f didapat dengan membalik rumus dari $S(p) = \frac{1}{f + \frac{(1-f)}{p}}$ menjadi $f = \frac{\frac{p}{S(p)} - 1}{p - 1}$ untuk setiap nilai $S(p)$ berdasarkan *strong scaling*, ditemukan rata-rata $f \approx 0.05$ (5%).

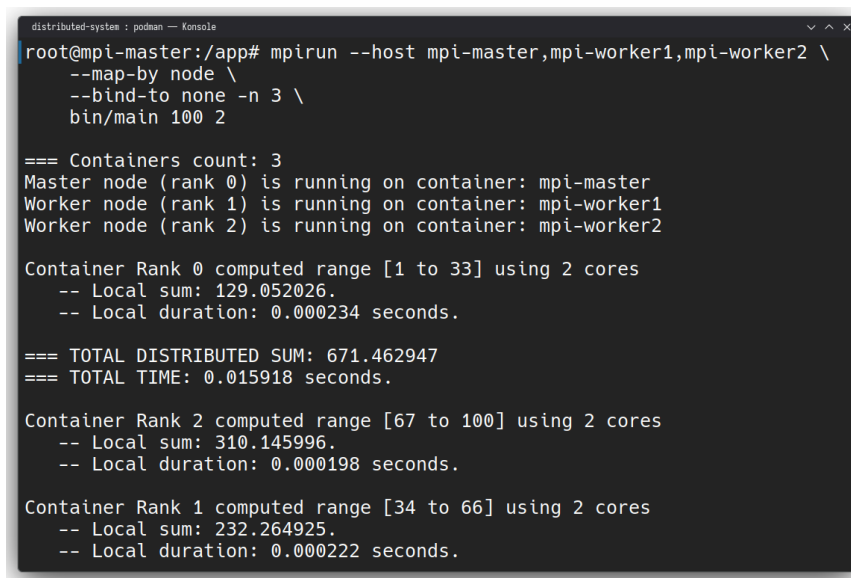


Berdasarkan grafik, terlihat *speedup* yang mendekati nilai ideal untuk *cores* 1 sampai 4, dengan efisiensi 95% - 100%. Terjadi penurunan ketika menggunakan 6 *cores*, *speedup* 5.58x dan efisiensi 93%. Prediksi *Amdahl* dengan serial *fraction* = 0.05 mulai terlihat ketika menggunakan 8 *cores* $S(8) = 5.83$ dengan $S_{amdahl}(8) = 5.92$, efisiensi menurun signifikan dari 93% (6 *cores*) menjadi 73%. Saat menggunakan 12 *cores*, *speedup* dapat mencapai 7.85x dengan efisiensi yang hanya 65%.

BAB VI

PEMBAHASAN

Implementasi komputasi paralel menggunakan OpenMP dan terdistribusi menggunakan OpenMPI dengan pola *master-worker* berhasil menunjukkan peningkatan performa yang signifikan dibandingkan eksekusi serial. *Worker* difokuskan hanya memproses dan mengirim hasil ke *master*. Hal ini dilakukan untuk menghindari keterlambatan ketika menampilkan output, terkadang hasil *worker* juga bisa selesai lebih cepat sehingga output tidak muncul berurutan. Dapat dilihat pada gambar dibawah ini yang menunjukkan output *mpi-worker2* dan *mpi-worker1* yang muncul terlambat setelah total seluruhnya ditampilkan.



```
distributed-system : podman -- Konsole
root@mpi-master:/app# mpirun --host mpi-master,mpi-worker1,mpi-worker2 \
--map-by node \
--bind-to none -n 3 \
bin/main 100 2

=== Containers count: 3
Master node (rank 0) is running on container: mpi-master
Worker node (rank 1) is running on container: mpi-worker1
Worker node (rank 2) is running on container: mpi-worker2

Container Rank 0 computed range [1 to 33] using 2 cores
-- Local sum: 129.052026.
-- Local duration: 0.000234 seconds.

=== TOTAL DISTRIBUTED SUM: 671.462947
=== TOTAL TIME: 0.015918 seconds.

Container Rank 2 computed range [67 to 100] using 2 cores
-- Local sum: 310.145996.
-- Local duration: 0.000198 seconds.

Container Rank 1 computed range [34 to 66] using 2 cores
-- Local sum: 232.264925.
-- Local duration: 0.000222 seconds.
```

Penambahan *workers* maupun *cores* terbukti efektif dalam memangkas waktu eksekusi, meskipun *speedup* tidak selalu linear dan nilai efisiensi yang menurun. *Overhead* terjadi di *processor (single-node)*, dapat dilihat dari hasil *weak scaling* untuk 1 *worker*, jumlah angka (*N*) yang diuji sama 1 miliar per *worker*. Namun, terdapat perbedaan waktu untuk 2 *cores* = 6.78 detik, sedangkan 4 *cores* = 6.84 detik, berbeda sekitar 0.06 detik.

BAB VII

KESIMPULAN

A. Kesimpulan

Berdasarkan hasil implementasi, pengujian, dan pembahasan yang telah dilakukan, dapat ditarik kesimpulan sebagai berikut:

1. Komputasi paralel terdistribusi terbukti mempercepat waktu pemrosesan secara signifikan. Berdasarkan data *strong scaling*, konfigurasi maksimal dengan 3 *workers* dan 4 *cores* per *worker* (total 12 *cores*) menghasilkan waktu eksekusi 4.32 detik untuk 5 miliar angka, dibandingkan 33.93 detik pada eksekusi serial dengan 1 *worker* dan 1 *core*, sehingga diperoleh *speedup* $7.85\times$
2. Penambahan *workers* dan *cores* memberikan peningkatan performa, namun laju peningkatannya tidak selalu linear. Hal ini sesuai dengan prediksi *Amdahl's Law*, di mana terdapat batas maksimal *speedup* akibat adanya bagian program yang tetap harus dijalankan secara serial sebesar 5% (0.05), seperti inisialisasi *MPI*, sinkronisasi, dan pengumpulan hasil menggunakan *MPI_Reduce*.
3. Prediksi *Amdahl* dengan rata-rata serial *fraction* = 0.05 mulai terlihat ketika menggunakan 8 *cores*, $S(8) = 5.83$ dengan $S_{amdahl}(8) = 5.92$, efisiensi menurun signifikan dari 93% (6 *cores*) menjadi 73%.

DAFTAR PUSTAKA

- [1] "Introduction to Parallel Computing," Lawrence Livermore National Laboratory, hosted by University of Arizona. [Online]. Available: https://hpcdocs.hpc.arizona.edu/events/workshop_materials/intro_to_parallel_computing/files/IntroToParallel.pdf
- [2] S. Zivanov, "CPU Threads vs. Cores: Differences Explained," phoenixNAP, Aug. 18, 2025. <https://phoenixnap.com/kb/cpu-threads-vs-cores>. (accessed May 30, 2026)
- [3] XTIVIA, "Sockets, Cores, and Threads, oh my!," Virtual-DBA, May 01, 2020. <https://virtual-dba.com/blog/sockets-cores-and-threads/>. (accessed May 30, 2026).
- [4] "Embarrassingly parallel explained," everything.explained.today, 2016. https://everything.explained.today/Embarrassingly_parallel/. (accessed May 30, 2026).
- [5] B. Barney, "Introduction to Parallel Computing Tutorial | HPC @ LLNL," hpc.llnl.gov. <https://hpc.llnl.gov/documentation/tutorials/introduction-parallel-computing-tutorial> (accessed May. 30, 2026).
- [6] M. Milanović, "Amdahl's Law," Laws of Software Engineering, Apr. 15, 2026. <https://lawsofsoftwareengineering.com/laws/amdahls-law/>. (accessed May 30, 2026)
- [7] "10.3. Parallel vs Serial Performance — CS160 Reader," computerscience.chemeketa.edu. <https://computerscience.chemeketa.edu/cs160Reader/ParallelProcessing/AmdahlsLaw.html> (accessed May 31, 2026).
- [8] M. van Steen and A. S. Tanenbaum, "A brief introduction to distributed systems," Computing, vol. 98, no. 10, pp. 967–1009, Aug. 2016, doi: <https://doi.org/10.1007/s00607-016-0508-7>

- [9] Performance model for hybrid MPI+OpenMP Master/Worker applications. (2015). Barcelona: TDX (Tesis Doctorals en Xarxa). Available::
<https://tdx.cat/bitstream/handle/10803/283403/acc1de1.pdf>
- [10] B. Barker, "Message Passing Interface (MPI)," 2015. Accessed: Jun. 02, 2026. [Online]. Available:
<https://cac.cornell.edu/Education/training/StampedeJan2015/IntroMPI.pdf>